

Gaussian Peak Fitting with labfit

A Practical Walkthrough Using Mercury Emission Spectra

LabFit

least-squares curve fitting for Python

built-in models

error propagation

publication plots



M. J. Doyle

08/06/2026

Covers: loading spectral data, Gaussian models, initial guesses, χ^2 , multi-peak fitting, and interpreting results.

Library: <https://github.com/matthewjdoyle/labfit>

Spectrum basics & the Gaussian model

Emission spectrum: intensity vs. wavelength. Each peak is an electronic transition in the atom. Line shapes are broadened by thermal motion (Doppler broadening).

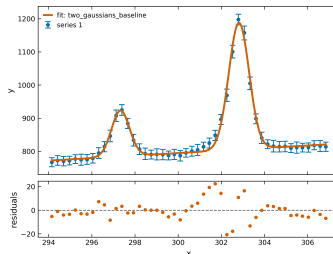
Gaussian:

$$I(\lambda) = A \exp \left[-\frac{1}{2} \left(\frac{\lambda - \lambda_0}{\sigma} \right)^2 \right]$$

Param	Meaning	Units
A	Peak amplitude	counts
λ_0	Centre wavelength	nm
σ	Width (σ)	nm

Why fit?

Noise shifts the apparent maximum; the true centre lies between pixels. Fitting uses *all* data points and gives rigorous uncertainties.



Gaussian with baseline

Real spectra sit on a background (stray light, dark current). Add a linear term:

$$I(\lambda) = A \exp \left[-\frac{1}{2} \left(\frac{\lambda - \lambda_0}{\sigma} \right)^2 \right] + m\lambda + b$$

Model in labfit	Parameters
gaussian_baseline	amplitude, mean, sigma, m, b

Workflow: Load CSV → Select peak region → Fit → Plot & interpret.

Step 1-2: Load data & select a peak region

```
from labfit.io import load_csv
from labfit import fit, plot_fit

data = load_csv("mercury_int10s.csv",
                x_col="wavelength_nm",
                y_col="intensity",
                y_err_col="y_error")
```

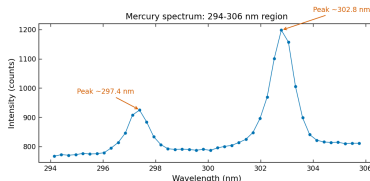
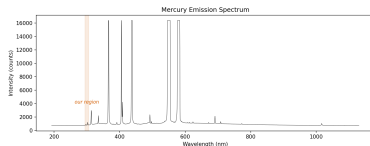
Output: 3648 pts, 192–1130 nm, max 16383 counts.

```
import numpy as np

mask = (data.x > 294) & (data.x < 307)
x_pk = data.x[mask]
y_pk = data.y[mask]
e_pk = data.y_err[mask]
```

Only ~48 points covering two small peaks.

CSV could also be loaded with `numpy.loadtxt` or `pandas.read_csv`; `labfit.io.load_csv` is a convenience wrapper that preserves column names and detects error columns.



Step 3: Define a custom two-Gaussian model

For overlapping peaks, write a model function or use the built-in equation:

$$I(\lambda) = A_1 e^{-(\lambda - \mu_1)^2 / 2\sigma_1^2} + A_2 e^{-(\lambda - \mu_2)^2 / 2\sigma_2^2} + m\lambda + b$$

```
def two_gaussians_baseline(x, a1, m1, s1, a2, m2, s2, m, b):  
    g1 = a1 * np.exp(-0.5 * ((x - m1) / s1) ** 2)  
    g2 = a2 * np.exp(-0.5 * ((x - m2) / s2) ** 2)  
    return g1 + g2 + m * x + b
```

```
result = fit(x_pk, y_pk, model=two_gaussians_baseline,  
            sigma=e_pk,  
            p0={"a1": 150, "m1": 297.4, "s1": 0.6,  
               "a2": 400, "m2": 302.8, "s2": 0.6,  
               "m": 0, "b": 780})
```

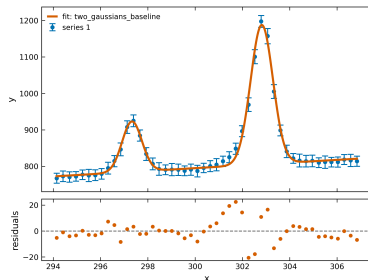
p0 provides initial guesses:

- a1, a2: approximate peak heights
- m1, m2: approximate centre wavelengths
- s1, s2: approximate widths (half-width at half-height $\times 0.85$)
- m, b: baseline slope and offset

Step 4: Interpret the fit results

FitResult: two_gaussians_baseline

a1 = 138.4 +/- 6.8
m1 = 297.33 +/- 0.024 nm
s1 = 0.418 +/- 0.025 nm
a2 = 382.6 +/- 6.8
m2 = 302.82 +/- 0.010 nm
s2 = 0.472 +/- 0.009 nm
m = 3.79 +/- 0.34
b = -342.9 +/- 103.3
reduced chi2 = 0.301
p = 1



Both peaks are well resolved: $\chi^2_\nu = 0.30$ (close to 1). Peak centres have sub-0.03 nm precision!

What χ^2 tells you

$$\chi^2_\nu = \frac{1}{\nu} \sum_i \left(\frac{y_i - f(x_i)}{\sigma_i} \right)^2$$

- $\chi^2_\nu \approx 1$: residuals are consistent with the uncertainties
- $\chi^2_\nu \gg 1$: model is wrong **or** errors are underestimated
- $\chi^2_\nu \ll 1$: errors may be overestimated

Our $\chi^2_\nu = 0.30$ is slightly below 1, suggesting our error bars are a bit conservative. This is still a **good fit** – the residuals scatter randomly around zero with no systematic pattern.

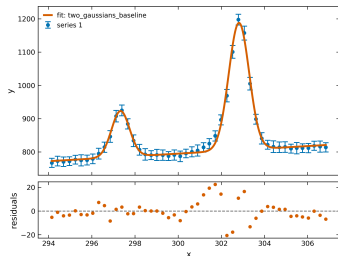
What if $\chi^2_\nu \gg 1$? Either the model is wrong (e.g. trying a single Gaussian on two overlapping peaks) or the uncertainties are too small. Always inspect the residuals to tell the difference!

Step 5: Visualise with residuals

```
from labfit import plot_fit

p = plot_fit(result, show_residuals=True)
p.save("mercury_300nm_bimodal_fit.png")
```

`plot_fit` produces data with error bars, the best-fit curve, and a **residual sub-panel** showing $(y_i - f(x_i))/\sigma_i$.



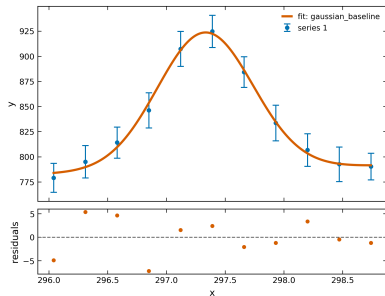
Alternative: fit each peak separately

You can also fit each peak individually with `gaussian_baseline`:

Peak at 297.3 nm:

```
mask1 = (data.x > 296) & (data.x < 299)
r1 = fit(data.x[mask1], data.y[mask1],
        model="gaussian_baseline",
        sigma=data.y_err[mask1])
```

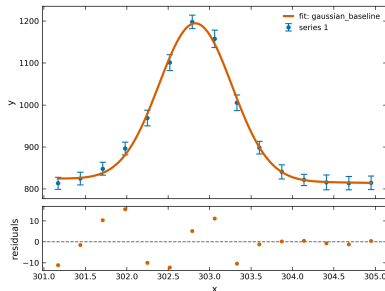
Amplitude = 137 ± 4 , Centre
= 297.33 ± 0.015 nm, $\chi^2_\nu = 0.10$



Peak at 302.8 nm:

```
mask2 = (data.x > 301) & (data.x < 305)
r2 = fit(data.x[mask2], data.y[mask2],
        model="gaussian_baseline",
        sigma=data.y_err[mask2])
```

Amplitude = 374 ± 8 , Centre
= 302.83 ± 0.012 nm, $\chi^2_\nu = 0.40$



Separate fits are simpler but ignore the overlap. Use a joint model when peaks are close enough to share baseline.

Practical tips

1. **Always plot first.** Know your data shape before choosing a model.
2. **Zoom in.** Fit one peak (or one small group) at a time.
3. **Give good p_0 .** The optimiser is not magic.
4. **Examine residuals.** Patterns mean your model is missing something.
5. $\chi^2 \gg 1$ can mean underestimated errors, not a bad fit.
6. **Read the uncertainties.** If error $>$ value, the parameter is unconstrained.
7. **Document everything.** Save the code, fit window, and parameter table – future you will thank present you.

Summary

You can now:

- Load spectral CSV data with `labfit.io.load_csv`
- Select a region of interest (NumPy masking)
- Fit Gaussian (+ baseline) to an emission line
- Define a custom multi-Gaussian model
- Interpret parameters, uncertainties, and χ^2
- Spot unreliable fits and know what to try next

5-line workflow:

```
from labfit.io import load_csv
from labfit import fit, plot_fit
data = load_csv("mercury_int10s.csv", x_col="wavelength_nm", y_col="intensity")
result = fit(data, model="gaussian_baseline")
plot_fit(result).save("my_fit.png")
```

Happy fitting! ★ <https://github.com/matthewjdoyle/labfit>